

Audit Research

Matteo Franzil, Valentino Armani, Luis Augusto Dias Knob, Domenico Siracusa

June 13, 2024

Audit Project

The scope of this project is to devise, implement, and evaluate a system that performs audit log parsing, (possibly) reduction, and generation of a graph (or similar structure) that captures the interactions between the events in audit logs.

Motivation

- Cloud Auditing is a powerful security feature that as it is does not realise its full potential. Indeed, some tools present in the market perform audit crunching and already provide some intelligence over the incoming information, but no one performs actual cross-log correlation between temporally distinct events. Furthermore, the such tools tend to be very verbose and generate a lot of false positives due to the inherent background noise of the cloud environment.
- Other academic works focus on doing data forensics and post-mortem investigation over system audit logs. However, we want to dynamically and in-real-time parse our log so that we can perform anomaly detection rather than letting security experts do it.
- GUI and querying are secondary: the graph should be quick to be generated and to parse, but in the end a machine will query it.
- To the best of our knowledge no one explored the capabilities of audit logging for doing real time anomaly detection over cloud environments.

Objectives

1. Parse audit logs, extracting relevant information and discarding redundant and non-informative data
2. Store the parsed data in a structured format that captures:
 - the relationships between the events and the entities involved
 - the temporal dimension of the events
 - the context in which the events occurred
3. Evaluate the performance of the system in terms of:
 - the correctness of the generated structure
 - we should investigate the loss of information and the choices that must be made between preserving data and discarding it
 - the effectiveness of the structured data
 - the information that can be extracted
 - the ease-of-use of the data structure and the preservation of causality relationships

- the efficiency of the parsing process, e.g.
 - the spatial and temporal complexity in traversing the data
 - the time required to parse a log file
 - the memory footprint of the system
 - its feasibility as a real-time system (not necessarily)

Timeline and milestones

(V) = Valentino, (M) = Matteo

Weeks 1-2 (May 7 - May 20)

- (M+V) Literature review
 - (audit) log parsing and analysis
 - graph-based data structures
 - data provenance
- (M+V) Initial investigation of K8s audit logs
 - Set up infrastructure to collect logs
 - Identify priority events to parse
 - * Assess possible problematic areas
- Results
 - A literature review (shared) document
 - A shared cluster with K8s running to collect logs from
 - Initial rough idea of events to focus on

Weeks 3-7 (May 21 - June 24)

- (M+V) Designing the system
 - Choosing which data structure to use
 - Outlining a definitive priority list of events to parse
 - * This also includes which Service Accounts to focus on first
 - Understanding how to include the temporal dimension
- (V) Initial implementation of parser for logs
 - Parse JSON logs
 - Extract relevant information
 - Build a data structure to store the parsed data
 - * Optimize for fast access and traversal
- (M) Work on understanding the relationships
 - Understand: when I do X, what is generated? Who is involved?
 - Which events are related to each other? Which can be discarded?
- Results
 - A shared document with the design of the system
 - An ordered list of *milestones* to reach, w.r.t. the capabilities of the system, along with the difficulties in doing so

Weeks 8-10 (June 25 - July 16)

- Continuation of the work from the previous weeks, step by step adding more features to the parser making it more capable of extracting information from the logs.
- Results
 - An early implementation of the parser that should already be able to extract some information, albeit not programmatically.

Weeks 11-12 (July 17 - July 30)

- (V) Focus on: querying the data structure
 - An user should be able to quickly find the information they are looking for.
 - * e.g. I put in a user and I can "follow" what the user did
 - * this possibly creates subgraphs that are "isolated" from the rest of the graph, on the fly
- (M) Initial assessment of the system
 - Correctness of the extracted information
 - Effectiveness of the structured data and its usability
 - Efficiency of the parsing process
 - In reality this should be a continuous process
- Results
 - A prototype of a system that is now queryable
 - A document outlining the evaluation process and the results obtained

Weeks 12-16 (July 30 - August 26)

- (V) Refining queries
 - Could be the right moment to start making *user stories* to understand how the system could be used and what features users would like to see
 - Strong focus on the temporal dimension
 - * Assume an incident happened at time X , what happened in timespan $[X - 1h, X + 1h]$?
 - * Is it possible to "replay" the events that led to the incident?
- (M) Work towards feature-completeness
 - The system should be able to parse all the events we set out to parse
 - Each event should be thoroughly tested, the choices made strongly justified and documented
- Did we meet the list of milestones we set up?

Yes Perfect, we can move on to the next phase

No Take a time machine and go back to the previous weeks to fix the issues

Weeks 17-18 (August 27 - September 9)

- (M+V) Starting the publication draft
 - The document should generalize the work done, making it suitable for not just Kubernetes logs but for any kind of audit log (e.g. AWS, GCP)
 - Still to be decided which type of publication we are aiming for
- (M+V) Final assessment of the system
 - The system should be able to parse logs in a reasonable amount of time, and the data structure should be efficient in storing the data
 - The system should be able to answer queries in a reasonable amount of time
 - The query system should be user-friendly, and the results should be easily interpretable
 - The system could have some kind of visualization to help users understand the data
- If something went wrong, this is the correct moment to apply fixes and improvements to the system.
- Results
 - A first WIP of the publication, still in the early stages of writing
 - A final assessment of the system
 - A visualization of the data structure
 - A final report on the project

Weeks 19-25 (September 10 - October 28)

- Writing the publication
- Finalizing the system
- Closure work
- Results
 - A publication
 - A final report on the project
 - A final version of the system

Misc notes

Motivation for the project

- Signature-based detection is not enough. To demonstrate so, we can use the example of Falco. Glossing over the nightmare that is installing it, Falco comes with a pre-set of rules that are fairly basic. For example, rules are triggered when SAs are created and deleted, secrets are accessed, and so on. However, these rules are exceedingly verbose and alerts are triggered for use cases that are not malicious at all. For example, Flannel renewing SA tokens, or the kube-controller-manager creating a secret for a SA when a namespace is created, are all examples of false positives that are triggered by Falco.

Behaviour of the audit logs

Generic

- `watch` requests are long-running and are usually made by control plane components. Since the `ResponseStarted` and `ResponseComplete` events can be in different times, we chose to **remove** the `ResponseStarted` event from the logs.
 - Indeed, for us it is important to have the `requestReceivedTimestamp` that is present in all events anyway. With that, we can understand when the request was made and when it was completed.
 - `ResponseStarted` appears in `watch` requests, in `kubectl exec` requests, and in `delete` requests. In the latter case, `ResponseComplete` is sent once the object is actually deleted.
- `kubectl` requests are internally translated into a set of API calls that are made by the `kube-apiserver`. Further subsections detail such calls and how they can help us derive templates for the parser.

Namespaces

- Every time a namespace is created, the following things happen:
 - A `list` request is made to `/api/v1/namespaces/{}/resourcequotas` by the `kube-apiserver` for no apparent reason
 - * To add insult to injury, usually this line is out of order in the log
 - The `kube-controller-manager` gets the `service-account-controller` SA and generates a token for it
 - The SA `root-ca-publisher` puts the `root-ca.crt` in a `ConfigMap` in the namespace
 - The `service-account-controller` creates a SA called `default` in the namespace
- When a namespace is deleted, all subresources are deleted with a `deletecollection` request followed by a `list`. Exceptions include `pods`, that weirdly are deleted by a `list`, then `deletecollection`, then `list` again. Also, when `configmaps` are deleted, the `kube-controller-manager` (and not the `root-ca-cert-publisher`) tries to create again the CM that was just deleted, but is rejected because the namespace is being terminated.

Other quirks

- When a secret is created for a SA the secret is instantly patched by the `kube-controller-manager`, which is in charge of actually generating that secret along with an object like

```
{"data":{"ca.crt":"...","namespace":"...","token":"..."}
```

where the three fields are base64 encoded.

- Secrets can be traced back to the Pod they have been mounted into by either looking at the spec or monitoring for `watch` requests by the node that is hosting the pod.
- Users with access to `kubectl exec` can access the secrets of the pods they are exec-ing into using `printenv`.
- `ResourceQuotas` can be used for more than just limiting resources, they can also be used to limit the number of objects that can be created. This also applies to things like `serviceaccounts`: thus, it seems that every time a potentially limitable object is created, a `get` request is made to `/api/v1/{object}` to assess the current state.

- When CSRs are being approved or denied, the **update** verb is used and the actual decision is in the payload of the request. This makes it less straightforward to understand if the request was allowed or denied.
- When old-style ServiceAccount tokens are used, instead of the new CSR style Users, the API server detects the *offending* action and adds a label `kubernetes.io/legacy-token-last-used` to the Secret.

Impersonation

- Users with access to the **impersonate** verb and one of **users**, **groups**, and **serviceaccounts** can impersonate other identities.
- This is correctly reflected in the audit logs, but could be a problem implementation-wise.

Control plane components

- Control plane components' "background noise" has been filtered out and the details are available in `queries.org`.

Future work

- Using KWOK
- Ask for a master's student to create a simulator with KWOK, defining scenarios and gathering the audit log